Experiment Title: Implementation of First Come, First Served CPU Scheduling Algorithm in C++

Topic: Non-Preemptive FCFS CPU Scheduling Simulation with Arrival Time in C++

1. Problem

Implement the non-preemptive First Come, First Served (FCFS) CPU scheduling algorithm in C++. Given the number of processes, arrival times (AT), and execution/burst times (BT), schedule the processes strictly by arrival order, handle CPU idle time, and compute Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and average metrics along with a Gantt chart.

2. Theoretical Solution

FCFS allocates the CPU to processes strictly in their order of arrival. Execution is non-preemptive: once a process gains the CPU, it runs to completion. If the CPU is idle when a process arrives, its start time equals its arrival time ($ST_i = \max(currentTime, AT_i)$).

2.1 Formulas

For each process i:

- $CT_i = ST_i + BT_i$
- $TAT_i = CT_i - AT_i$
- $WT_i = TAT_i - BT_i$
- $Avg_TAT = \frac{1}{n}\sum TAT_i, \quad Avg_WT = \frac{1}{n}\sum WT_i$

2.2 Algorithmic Steps

1. Read process count n, arrival times AT, and burst times BT.
2. Sort processes by arrival time using stable sort.
3. For each process, update $currentTime = \max(currentTime, AT_i)$, then compute CT_i, TAT_i, and WT_i.
4. Display the schedule table, average metrics, and Gantt chart sequence.

3. Expected Input/Output (I/O) Set

Expected input and scheduling results for $n = 4$ processes:

Process	Arrival (AT)	Execution (BT)	Start (ST)	Completion (CT)	Turnaround (TAT)	Waiting (WT)
P1	0	4	0	4	4	0
P2	1	3	4	7	6	3
P3	2	1	7	8	6	5
P4	3	2	8	10	7	5

Expected Averages: Average Waiting Time = 3.25, Average Turnaround Time = 5.75.

4. Source Code

Complete C++ implementation of the First Come, First Served scheduling algorithm:

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Process {
    int id;
    int arrival;
    int execution;
    int start;
    int completion;
    int turnaround;
    int waiting;
};

int main() {
    int n;

    cout << "Enter the number of processes: ";
    cin >> n;

    if (cin.fail() || n <= 0) {
        cout << "Invalid number of processes!\n";
        return 1;
    }

    vector<Process> processes(n);

    // Input process details
    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;

```

```
        cout << "Enter arrival and execution time for process "
             << processes[i].id << ": ";

        cin >> processes[i].arrival >> processes[i].execution;

        if (cin.fail() ||
            processes[i].arrival < 0 ||
            processes[i].execution <= 0) {
            cout << "Invalid input! Arrival time must be "
                 << "non-negative and execution time must "
                 << "be positive.\n";
            return 1;
        }
    }

    // Sort by arrival time; preserve input order for ties
    stable_sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.arrival < b.arrival;
        });

    int currentTime = 0;
    double totalWaiting = 0;
    double totalTurnaround = 0;

    // Calculate scheduling times
    for (Process& p : processes) {
        // Handle CPU idle time
        currentTime = max(currentTime, p.arrival);

        p.start = currentTime;
        p.waiting = p.start - p.arrival;

        currentTime += p.execution;

        p.completion = currentTime;
        p.turnaround = p.completion - p.arrival;

        totalWaiting += p.waiting;
        totalTurnaround += p.turnaround;
    }

    // Display results in original process order
    sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.id < b.id;
        });

    cout << "\nProcess\tArrival\tExecution\tStart\tCompletion"
         << "\tTurnaround\tWaiting\n";

    for (const Process& p : processes) {
        cout << "P" << p.id << "\t"
             << p.arrival << "\t"
             << p.execution << "\t\t"
             << p.start << "\t"
             << p.completion << "\t"
             << p.turnaround << "\t\t"
             << p.waiting << '\n';
    }

    // Display average times
    cout << fixed << setprecision(2);

    cout << "\nAverage Waiting Time: "
         << totalWaiting / n << '\n';

    cout << "Average Turnaround Time: "
         << totalTurnaround / n << '\n';

    // Display Gantt chart in FCFS execution order
    stable_sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.arrival < b.arrival;
        });

    cout << "\nGantt Chart:\n";

    for (const Process& p : processes) {
        if (p.start > 0 && p.start > p.arrival) {
            // The CPU may have been idle before this process.
            // Idle intervals are displayed below using timestamps.
        }

        cout << "| ";
```

```
            if (p.start > 0) {
                cout << "P" << p.id << " ";
            } else {
                cout << "P" << p.id << " ";
            }
        }
        cout << "|\n";

        // Print process start and completion timestamps
        if (!processes.empty()) {
            cout << processes[0].start;

            for (const Process& p : processes) {
                cout << "\t" << p.completion;
            }

            cout << '\n';
        }

        return 0;
    }
```

5. Execution of Code

The program was compiled with **g++** and executed in Linux terminal:

ratul@firefly: ~/OS-Lab — bash

```
ratul@firefly:~/OS-Lab$ g++ fcfs.cpp -o fcfs
ratul@firefly:~/OS-Lab$ ./fcfs
Enter the number of processes: 4
Enter arrival and execution time for process 1: 0 4
Enter arrival and execution time for process 2: 1 3
Enter arrival and execution time for process 3: 2 1
Enter arrival and execution time for process 4: 3 2
Process  Arrival  Execution  Start  Completion  Turnaround  Waiting
P1       0        4          0      4           4           0
P2       1        3          4      7           6           3
P3       2        1          7      8           6           5
P4       3        2          8      10          7           5
Average Waiting Time: 3.25
Average Turnaround Time: 5.75
Gantt Chart:
| P1 | P2 | P3 | P4 |
0    4    7    8    10
ratul@firefly:~/OS-Lab$ _
```

Verification: The execution output matches the expected completion times, turnaround times, waiting times, and Gantt chart sequence, confirming correct non-preemptive FCFS scheduling.